PROJECT GUIDE / WEB LAYER

# TOR Front-End Roadmap

FastAPI + Jinja2 + HTML/CSS/JS · A focused, step-by-step plan for the TOR leaderboard and player cards.

**PROJECT GOAL**
Build a professional TOR dashboard with a full-width ranked leaderboard (narrow Rank and number columns, flexible Name column). Clicking a player opens an overlay card with a headshot, raw data, component breakdown, and a date-range histogram.

## Starting point

The scoring engine and calibration config already exist and stay untouched. server.py has been generated but is not yet understood; that comes first. This roadmap covers only the web layer: serving data, templates, markup, styling, and interaction.

## Ground rules

| | |
|---|---|
| **01** | Read only the current step's docs. Try a minimal version yourself, then test it in the browser. |
| **02** | Make one change at a time. Commit to Git after each milestone. |
| **03** | Verified means you saw it work in the browser—not that code was generated. |
| **04** | No CSS frameworks (Tailwind, Bootstrap) or JS frameworks until plain versions are understood. |

## Build sequence / Foundations

**00**

### Understand server.py

Priority: First, before any HTML  Docs: FastAPI tutorial: First Steps, Path Parameters, Query Parameters

**Focus**

- What the app object, route decorator, and path function do.
- How Uvicorn runs the app and how a URL becomes a response.
- Where server.py calls the TOR engine, what data shape comes back, and what each route returns.

Milestone: Run the server, open every route in the browser, and explain what each route does and where its data comes from.

Note: If any line in server.py is unclear, mark it and ask. Do not build on code you cannot explain.

**01**

### Render data with Jinja2

Priority: Second  Docs: FastAPI: Templates · Jinja2: Template Designer Documentation (variables, loops, filters)

**Focus**

- Pass data from a route into a template; loop over leaderboard rows and print fields.
- Format numbers with filters (for example, one decimal place on the score).
- Use template inheritance (a base layout) only after one page works.

Milestone: A route renders a plain, unstyled HTML page showing the real top 25 players from the engine.

**02**

### HTML structure

Priority: Third  Docs: MDN HTML basics, table elements, and data-* attributes

**Focus**

- Build the page skeleton: head, body, header, main.
- Use a semantic table or grid of rows for the leaderboard.
- Attach a stable player ID to every row with a data- attribute; never identify a player by name alone.

Milestone: Clean, valid markup where each row carries a player ID you can read back later.

IMPLEMENTATION / LAYOUT + INTERACTION

# Make the page work, then make it feel finished.

Keep each milestone small enough to verify in the browser before moving forward.

## 03 CSS layout and visual design

Priority: Fourth · fixes the Streamlit table problem  Docs: MDN CSS basics, CSS Grid guide, box model, custom properties (CSS variables)

**Focus**

- Use fixed widths for Rank and numeric columns; use 1fr for Name.
- Set spacing, typography, and a dark theme with CSS variables.
- Add row hover/selected states and responsive behavior for narrow screens.

Milestone: The leaderboard spans the page width, keeps numeric columns narrow and Name flexible, and looks portfolio-ready.

Note: Define colors and sizes once as variables so later restyling is a one-file edit.

## 04 The player card as a native dialog

Priority: Fifth  Docs: MDN: , showModal(), close(), and ::backdrop

**Focus**

- Create a modal over the leaderboard while the page underneath stays unchanged.
- Support opening/closing, including Escape and clicking outside.
- Leading layout idea: image top-left, raw data bottom-left, histogram top-right, component breakdown bottom-right.

Milestone: Clicking a row opens a placeholder card over the page showing that row's player ID, and it closes cleanly.

## 05 JavaScript, only what is needed

Priority: Sixth  Docs: MDN JavaScript Guide: selecting DOM elements, events, fetch, async/await

**Focus**

- Listen for row clicks and read the player ID from the row.
- Call a FastAPI route such as /player/{id} with fetch.
- Fill the card with returned JSON; handle failed or empty responses.

Milestone: Clicking a row fetches that player's data and displays their name, TOR score, and component scores in the card.

**WORKING RULE**

Do not skip ahead to styling or interaction while the current layer is still unclear. A working, explainable minimal version is the target for each step.

IMPLEMENTATION / DATA + FINISHING

# Complete the card and make it reliable.

## 06 Headshots and card content

Priority: Seventh  Docs: Decide the image source before writing display code

**Focus**

- The engine must retain the batter ID so it can be mapped to a headshot.
- Provide a fallback image when no headshot exists.
- Display the component breakdown.

Milestone: The correct headshot appears on every card, and a missing image fails gracefully.

## 07 Histogram and date range

Priority: Eighth  Docs: Choose after Step 5 · likely Chart.js or Plotly.js via script tag, plus MDN date input

**Focus**

- The engine must retain game_date for each at-bat.
- Start with one histogram and one variable.
- Date inputs call a FastAPI route with start/end dates and redraw the chart; handle invalid ranges and empty results.

Milestone: Choosing a date range updates the card's histogram to show only that window.

Note: Filtering chart observations must never silently recalculate the official TOR score.

## 08 Performance and polish

Priority: Last  Docs: FastAPI: Static Files · Python functools.lru_cache or loading data once at server startup

**Focus**

- Serve CSS, JS, and images as static files.
- Load engine results once instead of on every request (the cold load was about 19 seconds).
- Add loading, empty, and error states, a page title, and a favicon.

Milestone: The page loads quickly, and every failure mode shows a clear message instead of a blank space.

# Recommended sequence

| STEP | TOPIC | FIRST MILESTONE |
|---|---|---|
| 0 | Understand server.py | Explain every route |
| 1 | Jinja2 templates | Real data on a plain page |
| 2 | HTML structure | Rows carry player IDs |
| 3 | CSS Grid | Sharp full-width leaderboard |
| 4 | | Click opens placeholder card |
| 5 | JavaScript + fetch | Card shows real player data |
| 6 | Headshots | Correct image per player |
| 7 | Histogram + dates | Date range updates chart |
| 8 | Static files + caching | Fast, polished, handles errors |

**BOOKMARKS / MOST USED FIRST**

1. HTML (MDN)
2. CSS (MDN)
3. Jinja2
4. FastAPI
5. JavaScript (MDN)

**YOUR FIRST MILESTONE**

Run server.py, visit every route in the browser, and explain what each one does. Then render the real leaderboard on a plain HTML page. Styling comes after that.